

Tokenisation

1. Σκοπός:

- Το αρχείο **Tokenization** προετοιμάζει τα δεδομένα για τους αλγορίθμους ταξινόμησης Naive Bayes, Random Forest και Logistic Regression.
- Μετατρέπει τα κείμενα (train/pos, test/pos, train/neg, test/neg) σε ξεχωριστές λέξεις ή συνδυασμούς λέξεων.

2. CsvFileReader

- Διαβάζει τα αρχεία με θετικές/αρνητικές κριτικές.
- Αποθηκεύει κάθε κριτική ως αντικείμενο Example μαζί με το label (θετικό ή αρνητικό)

3. RegexTokenizer

- Αφαιρεί σημεία στίξης και ειδικούς χαρακτήρες.
- Εντοπίζει συνδυασμούς λέξεων που αλλάζουν το νόημα (π.χ. not good → not_good, very good → very_good).

4. Vocabulary

- Καταγράφει τη συχνότητα εμφάνισης κάθε λέξης μέσω HashMap (word → αντικείμενο Word).
- Αποθηκεύει πληροφορίες για κάθε λέξη (π.χ. θετικές/αρνητικές εμφανίσεις, Information Gain – IG).
- Αφαιρεί τις **n** πιο συχνές και τις **k** πιο σπάνιες λέξεις (δεν συμβάλλουν αποτελεσματικά στην ταξινόμηση).
- Υπολογίζει το IG για κάθε λέξη που απομένει, διατηρώντας τις **m** λέξεις με το υψηλότερο IG (με τη βοήθεια της κλάσης SortByIG).
- Χρησιμοποιεί τη μέθοδο **vectorize** για να μετατρέπει κάθε κριτική σε διάνυσμα 0/1, ανάλογα με την παρουσία λέξεων στο τελικό λεξιλόγιο.

5. HashMap Lookup

- Κάθε λέξη αποθηκεύεται ως κλειδί σε HashMap για γρήγορη αναζήτηση και βελτίωση της απόδοσης.

6. VectorExporter

- Αποθηκεύει τα τελικά δεδομένα (διανύσματα, ετικέτες, χαρακτηριστικές λέξεις) σε αρχεία.
 - **exportVectorsToFile**: τα δυαδικά διανύσματα (0/1).
 - **exportLabelsToFile**: τις ετικέτες κειμένων (1 για θετικά, 0 για αρνητικά).
 - **exportFeatureWordsToFile**: το λεξιλόγιο που χρησιμοποιήθηκε στην κωδικοποίηση.

Αν τρέξουμε το πρόγραμμα μας, θα δούμε ότι τα 5 αρχεία που δημιουργήθηκαν είναι τα εξής:

train_vectors.txt: Αρχείο που περιέχει τα διανύσματα χαρακτηριστικών για το σύνολο εκπαίδευσης

0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

train_labels.txt: features.txt:

12495	1	2983: supposed
12496	1	2984: poorly
12497	1	2985: perfect
12498	1	2986: minutes
12499	1	2987: crap
12500	1	2988: poor
12501	0	2989: nothing
12502	0	2990: horrible
12503	0	2991: boring
12504	0	2992: wonderful
12505	0	2993: worse
12506	0	2994: stupid
12507	0	2995: terrible
		2996: excellent
		2997: awful
		2998: waste
		2999: worst

train_labels.txt: Αρχείο που περιέχει τις ετικέτες (labels) για το σύνολο εκπαίδευσης (θετικά ή αρνητικά).

features.txt: Αρχείο που περιέχει τις λέξεις του λεξιλογίου που χρησιμοποιήθηκαν στην επεξεργασία των δεδομένων εκπαίδευσης.

test_vectors.txt: Αρχείο που περιέχει τα διανύσματα χαρακτηριστικών για το σύνολο δοκιμής, χρησιμοποιώντας το λεξιλόγιο της εκπαίδευσης.

24998	0,
24999	0,
25000	0,

test_labels.txt: Αρχείο που περιέχει τις ετικέτες (labels) για το σύνολο δοκιμής.

2.....Σύγκριση αλγορίθμων

1. Αλγόριθμος Naive Bayes χρησιμοποιώντας πιθανότητες Bernoulli σε σύγκριση με τον αλγόριθμο Sklearn Bernoulli

Δουλέψαμε με ένα σύνολο δεδομένων **50.000 κριτικών**, χωρισμένο εξίσου σε **θετικές και αρνητικές γνώμες**. Για να διασφαλίσουμε ότι το μοντέλο μας εκπαιδεύτηκε και αξιολογήθηκε σωστά, διαιρέσαμε τα δεδομένα ως εξής:

- **Σύνολο train (80% από 25.000 παραδείγματα) → 20.000 παραδείγματα** χρησιμοποιήθηκαν για την εκπαίδευση του μοντέλου.
- **Σύνολο development (20% από 25.000 παραδείγματα) → 5.000 παραδείγματα** χρησιμοποιήθηκαν για τη βελτίωση του μοντέλου και τον έλεγχο της γενίκευσης.
- **Σύνολο test (όλα τα 25.000 παραδείγματα) → Χρησιμοποιήθηκε για την τελική αξιολόγηση.**

Υπερπαράμετροι

Για να μετατρέψουμε τα κείμενα σε διανύσματα 0,1, εφαρμόσαμε:

- **k = 0.04** → Αφαιρέσαμε το **4% των λιγότερο συχνών λέξεων**.
- **n = 150** → Εξαίρεσαμε τις **150 πιο συχνές λέξεις**.
- **m = 3000** → Κρατήσαμε τις **3.000 πιο σημαντικές λέξεις** βάσει **Information Gain (IG)**
- **Binary (1/0 vectors)**

Επιπλέον, χρησιμοποιήσαμε **εξομάλυνση Laplace ($\alpha = 1.0$)** για να αποτρέψουμε τις μηδενικές πιθανότητες και να βελτιώσουμε την ακρίβεια του μοντέλου.

Αποτελέσματα από το Μοντέλο Naïve Bayes χρησιμοποιώντας Java για τον αλγόριθμο και numpy για την απεικόνιση

Μετά την εκπαίδευση και τη δοκιμή του μοντέλου μας **Bernoulli Naïve Bayes**, λάβαμε:

- **Macro-Averaged F1 Score: 0.8483**
- **Micro-Averaged F1 Score: 0.8485**
- **Καλή ισορροπία μεταξύ precision και recall**, που σημαίνει ότι **γενικεύει καλά**.
- **Καμπύλη Μάθησης:** Καθώς αυξήσαμε τα δεδομένα εκπαίδευσης, η απόδοση του μοντέλου στο σύνολο εκπαίδευσης μειώθηκε ελαφρώς, αλλά η απόδοσή του στο σύνολο ανάπτυξης βελτιώθηκε. Αυτό δείχνει ότι το μοντέλο μας απέφυγε το **overfitting** και μάθαινε αποτελεσματικά.

=== Learning Curve ===						
TrainSize	TrainPrec	TrainRec	TrainF1	DevPrec	DevRec	DevF1
2000	0.8938	0.9161	0.9048	0.8699	0.8629	0.8664
4000	0.8914	0.8941	0.8928	0.8771	0.8577	0.8673
6000	0.8951	0.8703	0.8825	0.8845	0.8581	0.8711
8000	0.8892	0.8718	0.8804	0.8831	0.8621	0.8725
10000	0.8925	0.8711	0.8817	0.8830	0.8641	0.8734
12000	0.8916	0.8728	0.8821	0.8825	0.8661	0.8742
14000	0.8891	0.8719	0.8804	0.8848	0.8702	0.8774
16000	0.8903	0.8725	0.8813	0.8832	0.8690	0.8760
18000	0.8893	0.8706	0.8798	0.8838	0.8681	0.8759
20000	0.8900	0.8718	0.8808	0.8836	0.8690	0.8762

```
=== Training Final Model on Full Train Set ===  
Evaluating on Test Data...
```

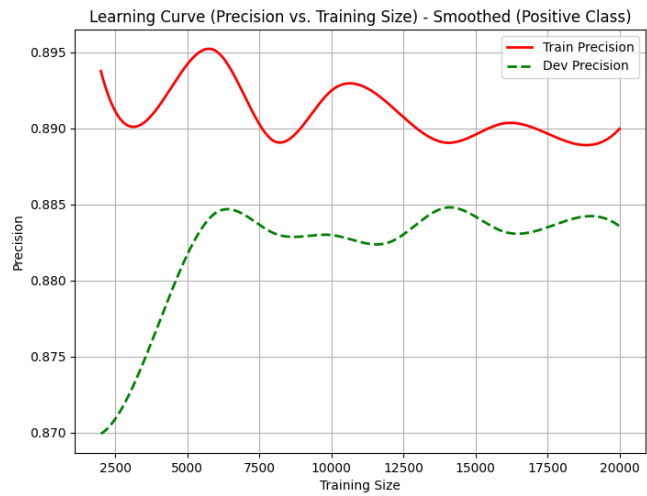
```
=== Final Results on Test Set ===  
Precision (Positive Class): 0.8695  
Recall (Positive Class):    0.8173  
F1 (Positive Class):       0.8426
```

```
Precision (Negative Class): 0.8276  
Recall (Negative Class):    0.8774  
F1 (Negative Class):       0.8518
```

```
Macro-Averaged Precision: 0.8486  
Macro-Averaged Recall:    0.8473  
Macro-Averaged F1:       0.8472
```

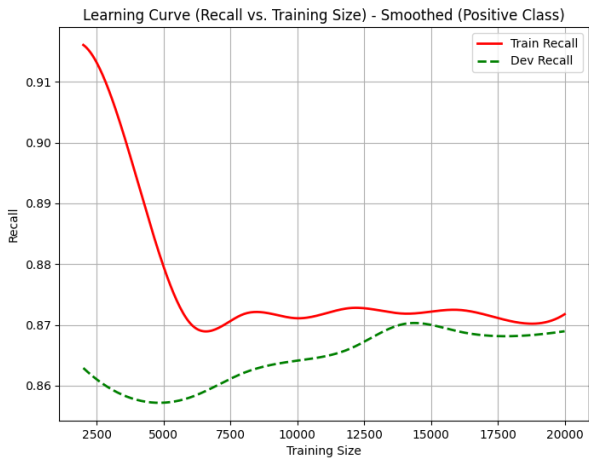
```
Micro-Averaged Precision: 0.8473  
Micro-Averaged Recall:    0.8473  
Micro-Averaged F1:       0.8473
```

Figure 1



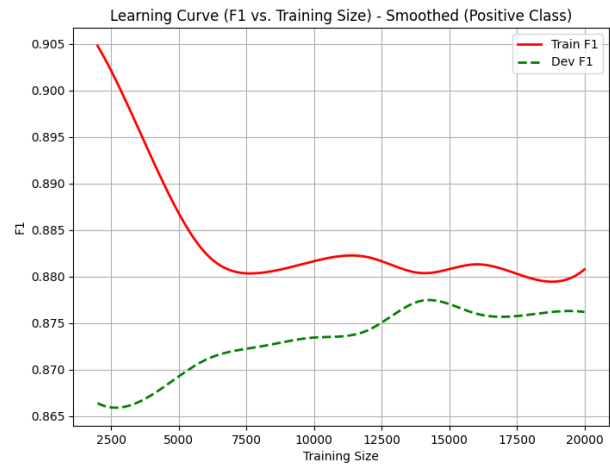
Navigation icons: Home, Back, Forward, Zoom In, Zoom Out, Full Screen, Save

Figure 1



Navigation icons: Home, Back, Forward, Zoom In, Zoom Out, Full Screen, Save

Figure 1



Navigation icons: Home, Back, Forward, Zoom In, Zoom Out, Full Screen, Save

Σύγκριση με το Bernoulli Naïve Bayes του Sklearn

Δοκιμάσαμε το **BernoulliNB του Scikit-learn** χρησιμοποιώντας τα ίδια features και vectors:

```
=== Sklearn BernoulliNB Results ===
Precision (Positive Class): 0.8708
Recall (Positive Class): 0.8189
F1 (Positive Class): 0.8440

Precision (Negative Class): 0.8291
Recall (Negative Class): 0.8785
F1 (Negative Class): 0.8531

Precision (Positive Class): 0.8708
Recall (Positive Class): 0.8189
F1 (Positive Class): 0.8440

Precision (Negative Class): 0.8291
Recall (Negative Class): 0.8785
F1 (Negative Class): 0.8531

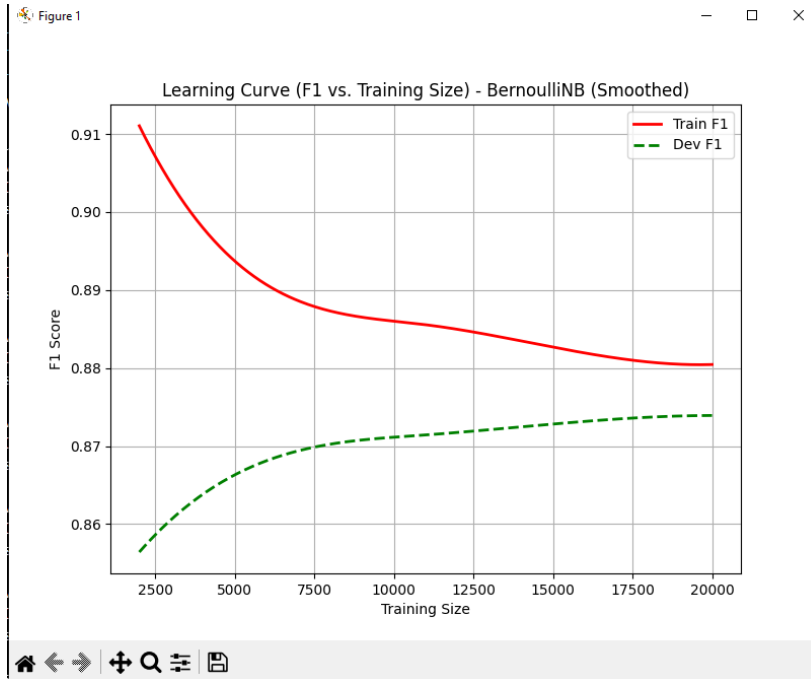
Precision (Positive Class): 0.8708
Recall (Positive Class): 0.8189
F1 (Positive Class): 0.8440

Precision (Negative Class): 0.8291
Recall (Negative Class): 0.8785
F1 (Negative Class): 0.8531

Macro-Averaged Precision: 0.8499
Macro-Averaged Recall: 0.8487
Macro-Averaged F1: 0.8485

Micro-Averaged Precision: 0.8487
Micro-Averaged Recall: 0.8487
Micro-Averaged F1: 0.8487
```

- **Macro-Averaged F1 Score: 0.8485** (Sklearn)
- **Micro-Averaged F1 Score: 0.8487** (Sklearn)
- Οι καμπύλες μάθησης.



Συμπέρασμα

- Το δικό μας μοντέλο **Naïve Bayes** είχε σχεδόν **την ίδια απόδοση** με την υλοποίηση του Sklearn..
- Οι **καμπύλες μάθησης** έδειξαν ότι και τα δύο μοντέλα **γενίκευσαν καλά** και **δεν είχαμε σοβαρό overfitting**.

2. Αλγόριθμος Λογιστικής Παλινδρόμησης σε σύγκριση με τον Αλγόριθμο Λογιστικής Παλινδρόμησης του Sk-learn

- **Σύνολο train (80% από 25.000 παραδείγματα) → 20.000 παραδείγματα** χρησιμοποιήθηκαν για την εκπαίδευση του μοντέλου.
- **Σύνολο development (20% από 25.000 παραδείγματα) → 5.000 παραδείγματα** χρησιμοποιήθηκαν για τη βελτίωση του μοντέλου και τον έλεγχο της γενίκευσης.
- **Σύνολο test (όλα τα 25.000 παραδείγματα) → Χρησιμοποιήθηκε** για την τελική αξιολόγηση.

Υπερπαράμετροι

Epochs: 80

Learning rate (h): 0.015

Regularization parameter (λ): 0.015

Decay Rate: 0.015

Η φόρμουλα για τη φθίνουσα ταχύτητα εκμάθησης είναι η εξής:

$$h = \frac{h_{initial}}{1 + decayRate * epochs}$$

Αποτελέσματα από το Μοντέλο Logistic Regression χρησιμοποιώντας Java για τον αλγόριθμο και numpy για την απεικόνιση

- **Macro-Averaged F1 Score: 0.8742**
- **Micro-Averaged F1 Score: 0.8742**

Αν το dataset είναι ισορροπημένο και οι δύο κλάσεις έχουν τον ίδιο αριθμό δειγμάτων, τότε η micro-μέση τιμή, που βασίζεται στα συνολικά TP, FP και FN, θα βγει ίδια.

```
=== Learning Curve ===
TrainSize | TrainPrec | TrainRec | TrainF1 | DevPrec | DevRec | DevF1
2000 | 0.9826 | 0.9864 | 0.9845 | 0.8213 | 0.8534 | 0.8370
4000 | 0.9691 | 0.9672 | 0.9681 | 0.8559 | 0.8638 | 0.8598
6000 | 0.9576 | 0.9601 | 0.9589 | 0.8573 | 0.8666 | 0.8619
8000 | 0.9489 | 0.9484 | 0.9487 | 0.8727 | 0.8730 | 0.8729
10000 | 0.9388 | 0.9510 | 0.9449 | 0.8667 | 0.8907 | 0.8785
12000 | 0.9354 | 0.9490 | 0.9421 | 0.8719 | 0.8943 | 0.8830
14000 | 0.9283 | 0.9474 | 0.9377 | 0.8741 | 0.9064 | 0.8899
16000 | 0.9318 | 0.9415 | 0.9366 | 0.8781 | 0.9000 | 0.8889
18000 | 0.9269 | 0.9387 | 0.9328 | 0.8787 | 0.9080 | 0.8931
20000 | 0.9263 | 0.9374 | 0.9318 | 0.8872 | 0.9068 | 0.8969
Learning curve exported to: C:\Users\ernes\Desktop\PartA\learning_curve_logistic.csv
```

```
=== Training Final Model on Full Train Set ===
Evaluating on Test Data...
```

```
=== Final Results on Test Set ===
```

```
Precision (Positive Class): 0.8732
```

```
Recall (Positive Class): 0.8736
```

```
F1 (Positive Class): 0.8734
```

```
Precision (Negative Class): 0.8735
```

```
Recall (Negative Class): 0.8731
```

```
F1 (Negative Class): 0.8733
```

```
Macro-Averaged Precision: 0.8734
```

```
Macro-Averaged Recall: 0.8734
```

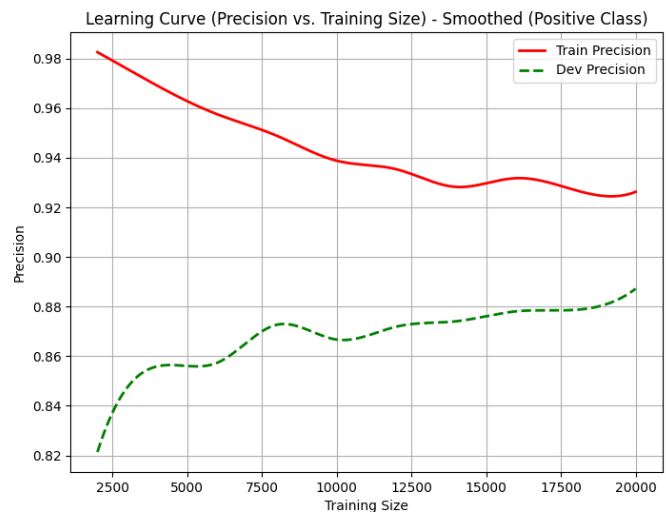
```
Macro-Averaged F1: 0.8734
```

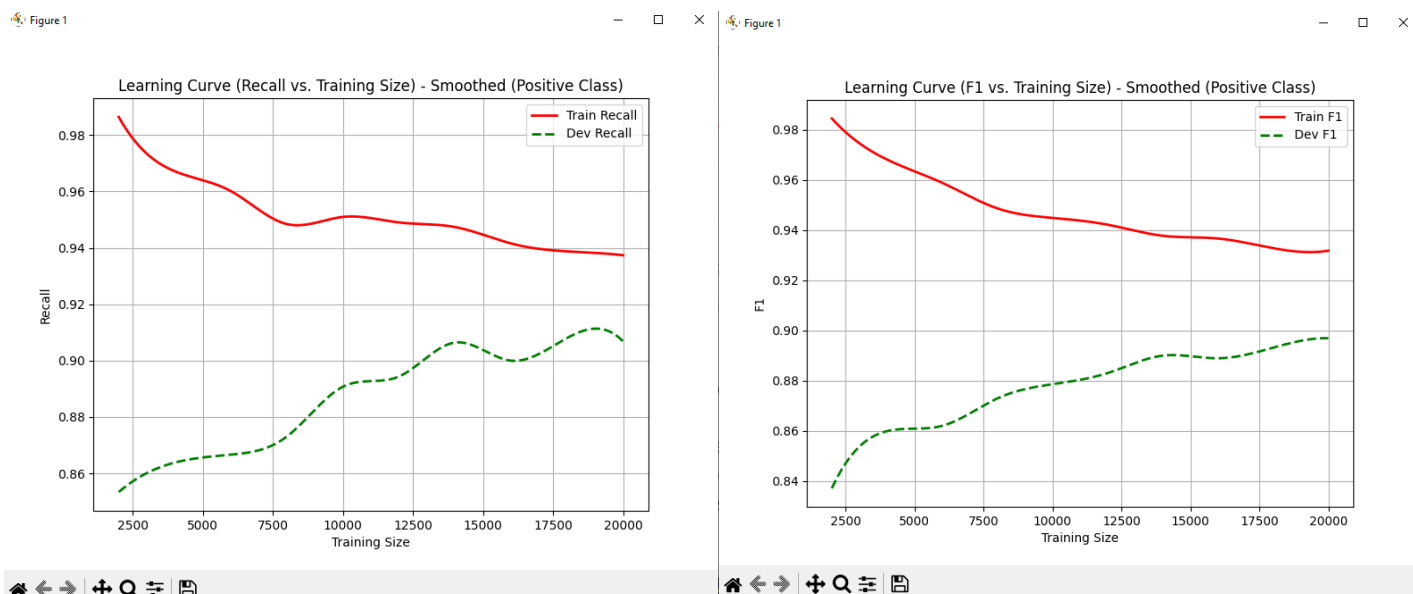
```
Micro-Averaged Precision: 0.8734
```

```
Micro-Averaged Recall: 0.8734
```

```
Micro-Averaged F1: 0.8734
```

Figure 1





Σύγκριση με το Logistic Regression του Sklearn

```

=== Sklearn LogisticRegression Results ===
Precision (Positive Class): 0.8690
Recall (Positive Class): 0.8668
F1 (Positive Class): 0.8679

Precision (Negative Class): 0.8671
Recall (Negative Class): 0.8693
F1 (Negative Class): 0.8682

Precision (Positive Class): 0.8690
Recall (Positive Class): 0.8668
F1 (Positive Class): 0.8679

Precision (Negative Class): 0.8671
Recall (Negative Class): 0.8693
F1 (Negative Class): 0.8682

Precision (Positive Class): 0.8690
Recall (Positive Class): 0.8668
F1 (Positive Class): 0.8679

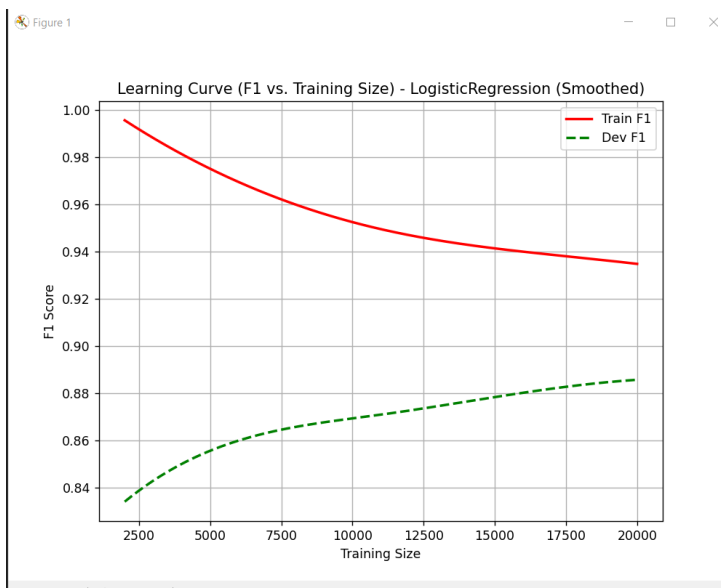
Precision (Negative Class): 0.8671
Recall (Negative Class): 0.8693
F1 (Negative Class): 0.8682

Macro-Averaged Precision: 0.8680
Macro-Averaged Recall: 0.8680
Macro-Averaged F1: 0.8680

Micro-Averaged Precision: 0.8680
Micro-Averaged Recall: 0.8680
Micro-Averaged F1: 0.8680

```

- **Macro-Averaged F1 Score: 0.8680** (Sklearn)
- **Micro-Averaged F1 Score: 0.8680** (Sklearn)
- Οι καμπύλες μάθησης ήταν σχεδόν ίδιες.



Συμπέρασμα

- Το μοντέλο έχει υψηλό variance, αφού η απόδοση στο train set είναι πολύ καλύτερη από το dev set. Η χρήση **περισσότερων δεδομένων** ή regularization θα μπορούσε να βοηθήσει

3. Αλγόριθμος Τυχαίο Δάσος (Random Forest) χρησιμοποιώντας τον αλγόριθμο ID3 σε σύγκριση με τον (Random Forest) του Sk-learn

- **Σύνολο train (85% από 25.000 παραδείγματα) → 21.250 παραδείγματα** χρησιμοποιήθηκαν για την εκπαίδευση του μοντέλου.
- **Σύνολο development (15% από 25.000 παραδείγματα) → 3750 παραδείγματα** χρησιμοποιήθηκαν για τη βελτίωση του μοντέλου και τον έλεγχο της γενίκευσης.
- **Σύνολο test (όλα τα 25.000 παραδείγματα) → Χρησιμοποιήθηκε** για την τελική αξιολόγηση.

Υπερπαράμετροι

- **Αριθμός Δέντρων :** 150
- Μέγιστα Χαρακτηριστικά Ανά Διαίρεση: $\text{sqrt}(3000) = 54.77 = \text{int}(54)$
- Μέγιστο Βάθος Δέντρου: 90%
- Κατώφλι Ομοιογένειας (Purity Threshold) = 90%
- Ελάχιστος Αριθμός Παραδειγμάτων Ανά Φύλλο: 1
- Κριτήριο Επιλογής Χαρακτηριστικών: IG Information Gain
- Μέθοδος Ψηφοφορίας: Πλειοψηφική ψήφος

Αποτελέσματα από το Τυχαίο Δάσος χρησιμοποιώντας Java για τον αλγόριθμο και numpy για την απεικόνιση

- **Macro-Averaged F1 Score: 0.8377**
- **Micro-Averaged F1 Score: 0.8377**

```
Train size: 21250
Dev size: 3750
Train size: 21250
Dev size: 3750
Test size: 25000
Dev positives: 1840, Dev negatives: 1910
Dev positives: 10660, Dev negatives: 10590

=== Learning Curve ===
TrainSize | TrainPrec | TrainRec | TrainF1 | DevPrec | DevRec | DevF1
2000 | 0.9813 | 0.9930 | 0.9871 | 0.8189 | 0.8087 | 0.8138
4000 | 0.9801 | 0.9870 | 0.9835 | 0.8369 | 0.8201 | 0.8284
6000 | 0.9781 | 0.9774 | 0.9778 | 0.8438 | 0.8103 | 0.8267
8000 | 0.9764 | 0.9761 | 0.9763 | 0.8429 | 0.8250 | 0.8338
10000 | 0.9736 | 0.9773 | 0.9754 | 0.8415 | 0.8337 | 0.8376
12000 | 0.9732 | 0.9761 | 0.9747 | 0.8373 | 0.8364 | 0.8369
14000 | 0.9723 | 0.9767 | 0.9745 | 0.8431 | 0.8413 | 0.8422
16000 | 0.9694 | 0.9759 | 0.9727 | 0.8427 | 0.8446 | 0.8436
18000 | 0.9697 | 0.9737 | 0.9717 | 0.8459 | 0.8353 | 0.8406
20000 | 0.9698 | 0.9715 | 0.9706 | 0.8461 | 0.8397 | 0.8429
Learning curve exported to: C:\Users\ernes\Desktop\PartA\learning_curve_RandomForest.csv
```



```

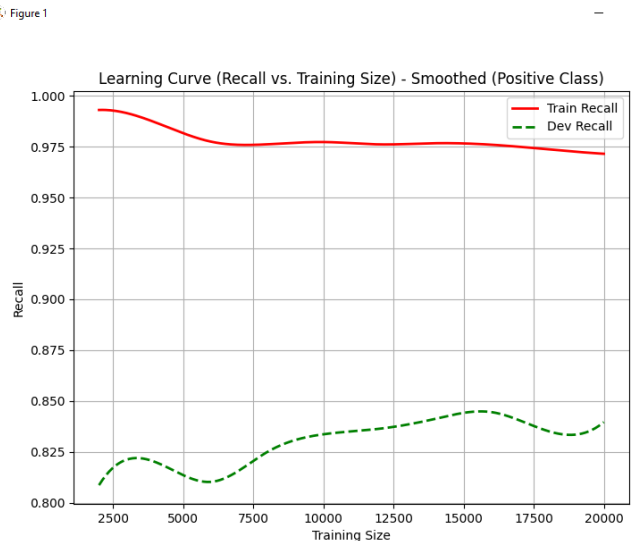
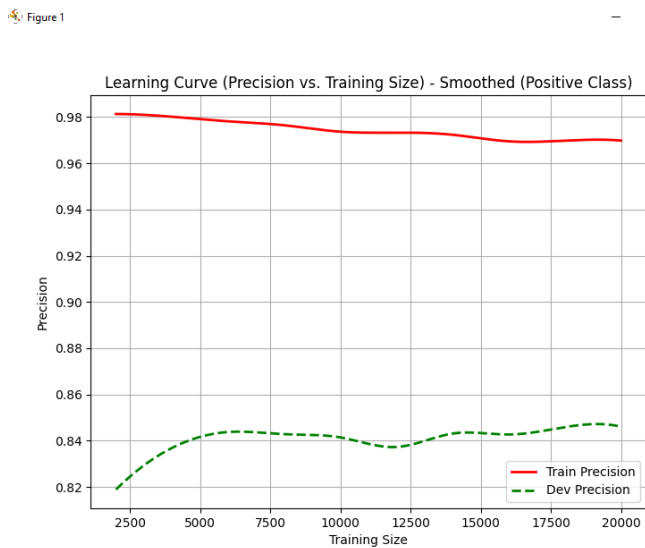
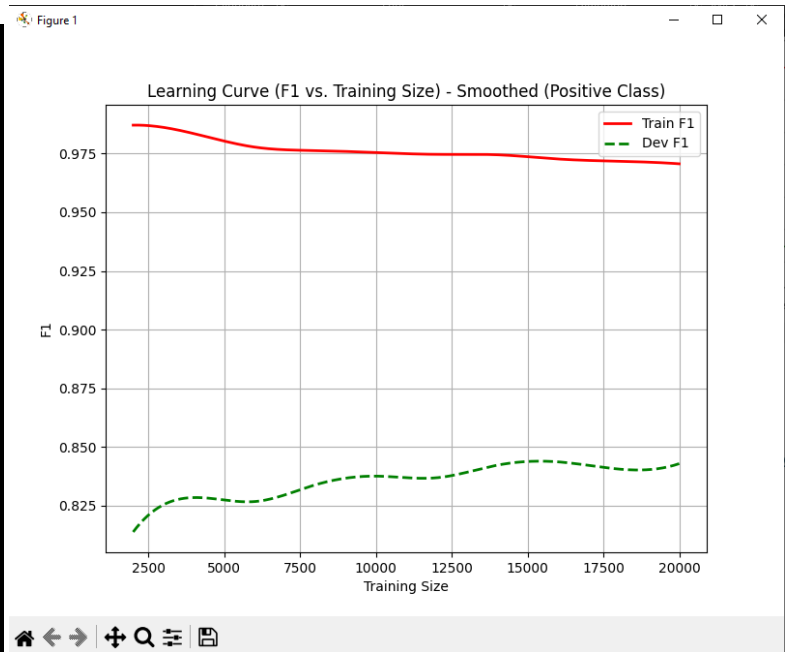
=== Final Results on Test Set ===
Precision (Positive Class): 0.8423
Recall (Positive Class):    0.8310
F1 (Positive Class):       0.8366

Precision (Negative Class): 0.8333
Recall (Negative Class):    0.8444
F1 (Negative Class):       0.8388

Macro-Averaged Precision: 0.8378
Macro-Averaged Recall:    0.8377
Macro-Averaged F1:       0.8377

Micro-Averaged Precision: 0.8377
Micro-Averaged Recall:    0.8377
Micro-Averaged F1:       0.8377

```



Σύγκριση με το Τυχαίο Δάσος του Sklearn

- **Macro-Averaged F1 Score: 0.8462** (Sklearn)
- **Micro-Averaged F1 Score: 0.8462** (Sklearn)
- Οι καμπύλες μάθησης ήταν σχεδόν ίδιες.

```

=== Sklearn RandomForest Results ===
Precision (Positive Class): 0.8499
Recall (Positive Class): 0.8354
F1 (Positive Class): 0.8426

Precision (Negative Class): 0.8382
Recall (Negative Class): 0.8525
F1 (Negative Class): 0.8453

Precision (Positive Class): 0.8499
Recall (Positive Class): 0.8354
F1 (Positive Class): 0.8426

Precision (Negative Class): 0.8382
Recall (Negative Class): 0.8525
F1 (Negative Class): 0.8453

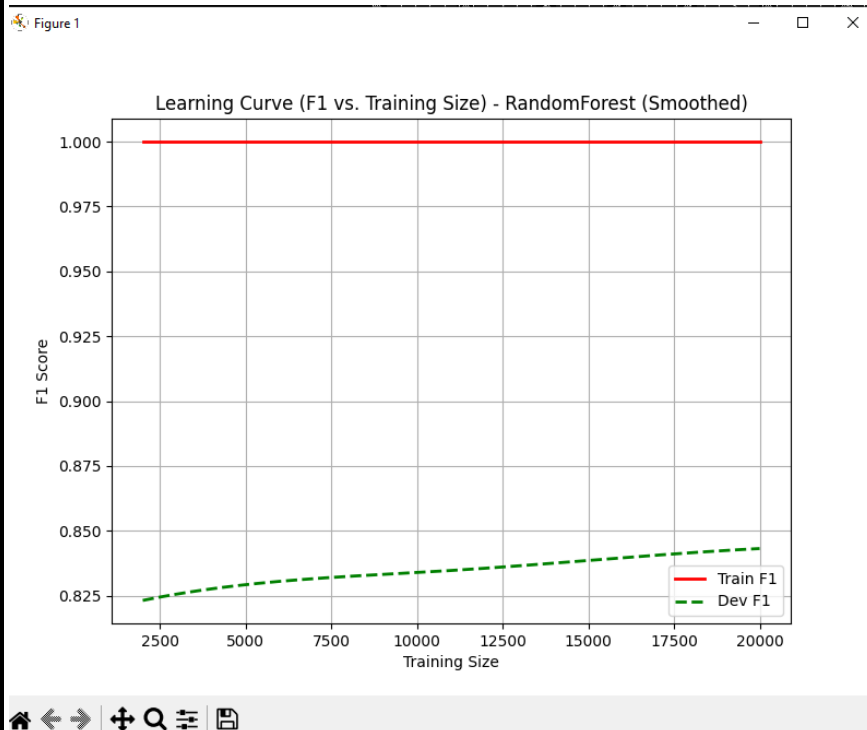
Precision (Positive Class): 0.8499
Recall (Positive Class): 0.8354
F1 (Positive Class): 0.8426

Precision (Negative Class): 0.8382
Recall (Negative Class): 0.8525
F1 (Negative Class): 0.8453

Macro-Averaged Precision: 0.8441
Macro-Averaged Recall: 0.8440
Macro-Averaged F1: 0.8439

Micro-Averaged Precision: 0.8440
Micro-Averaged Recall: 0.8440
Micro-Averaged F1: 0.8440

```

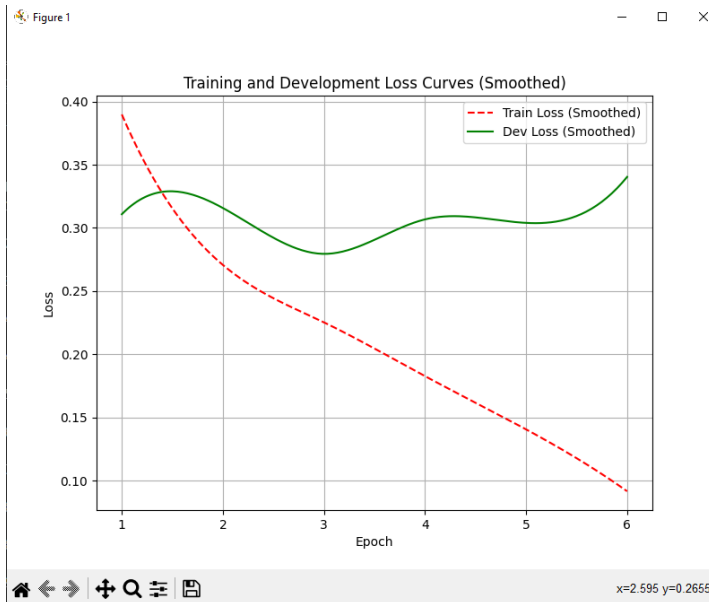


4. BiDirectional Stacked RNN χρησιμοποιώντας Pytorch and Adam Booster with ready embedded words from Glove

Υπερπαράμετροι

- **Στρατηγική διαίρεσης δεδομένων:**
 - Εκπαίδευση: 80%
 - Ανάπτυξη (validation): 20%
 - Τεστ: Ξεχωριστό dataset
- **Πολιτική early stopping:** (σημαίνει ότι αν το validation loss δεν βελτιωθεί για 3 συνεχόμενες εποχές, η εκπαίδευση σταματάει για να μην γίνει (overfitting). Παράλληλα, κάθε φορά που το validation loss βελτιώνεται, το καλύτερο μοντέλο σώζεται αυτόματα.)
 - (Patience): 3 epochs
 - Save του καλύτερου μοντέλου αν βελτιωθεί το validation loss for each epoch
- Μέγεθος των GloVe embeddings: 300
- Κατάψυξη προεκπαιδευμένων embeddings: Ναι (freeze_embeddings = True)
- Μέγιστο μήκος ακολουθίας (max_length): 500
- Μέγεθος του λεξιλογίου (vocab_size): 20.000
- Μέγεθος του batch (batch_size): 32
- Πλήθος επιπέδων (num_layers): 2
- Μέγεθος κρυφών καταστάσεων LSTM (hidden_dim): 256
- Αριθμός κλάσεων εξόδου (num_classes): 2 (δυαδική ταξινόμηση positive/negative)
- Dropout: 0.3
- Συνάρτηση απώλειας (loss function): CrossEntropyLoss

- **Activate function:** ArgMax (two categories)
- **Βελτιστοποιητής (optimizer):** Adam
- **Μάθημα μάθησης (learning rate):** 1e-3
- **Συντελεστής κανονικοποίησης L2 (weight decay):** 1e-5



Accuracy: 0.89868

Classwise Precision, Recall, F1-score:
Precision: [0.88396641 0.91456618]
Recall: [0.91784 0.87952]
F1-Score: [0.9005848 0.89670079]

Macro Average:
Precision: 0.8992662915661005
Recall: 0.8986799999999999
F1-Score: 0.8986427912010495

Micro Average:
Precision: 0.89868
Recall: 0.89868
F1-Score: 0.89868

Η σύγκριση όλων των αλγορίθμων μαζί

Algorithm	Precision Positive	Recall Positive	F1 Positive	Precision Negative	Recall Negative	F1 Negative	Macro Precision	Macro Recall	Macro F1	Micro Precision	Micro Recall	Micro F1
Naïve Bayes	0.8695	0.8173	0.8426	0.8276	0.8774	0.8518	0.8486	0.8473	0.8472	0.8473	0.8473	0.8473
Sklearn N.Bayes	0.8708	0.8189	0.8440	0.8291	0.8785	0.8531	0.8499	0.8487	0.8485	0.8487	0.8487	0.8487
Logistic Regression	0.8746	0.8738	0.8742	0.8739	0.8739	0.8747	0.8743	0.8742	0.8783	0.8783	0.8783	0.8783
Sklearn L.Regression	0.8690	0.8668	0.8679	0.8671	0.8693	0.8682	0.8680	0.8680	0.8680	0.8680	0.8680	0.8680
Random Forest	0.8423	0.8310	0.8366	0.8333	0.8444	0.8388	0.8378	0.8377	0.8377	0.8377	0.8377	0.8377
Sklearn Random Forest	0.8499	0.8354	0.8426	0.8382	0.8525	0.8453	0.8441	0.8440	0.8439	0.8440	0.8440	0.8440
BiRNN LSTM cells ADAM opt	0.8839	0.9178	0.9005	0.9145	0.8795	0.8967	0.8992	0.8986	0.8986	0.8986	0.8986	0.8986